

KONCEPT PROJEKTU I SPECYFIKACJA FUNKCJONALNA

Gra: Spider Solitaire SFML

Gra: Spider Solitaire SFML

Skład grupy:

Imię i Nazwisko: Maciej Popławski , Piotr Madelski

Nr Indeksu: 167115 , 168041

Repozytorium GitHub:

https://github.com/maciejpoplawski24/Projekt_zaliczeniowy_cpp.git

1. Prezentacja aplikacji

Aplikacja jest komputerową adaptacją klasycznej gry karcianej Pasjans Pająk w wersji jednokolorowej. Celem gracza jest uporządkowanie talii kart w kolumnach roboczych poprzez układanie sekwencji malejących od Króla do Asa.

Elementy graficzne (Widżety):

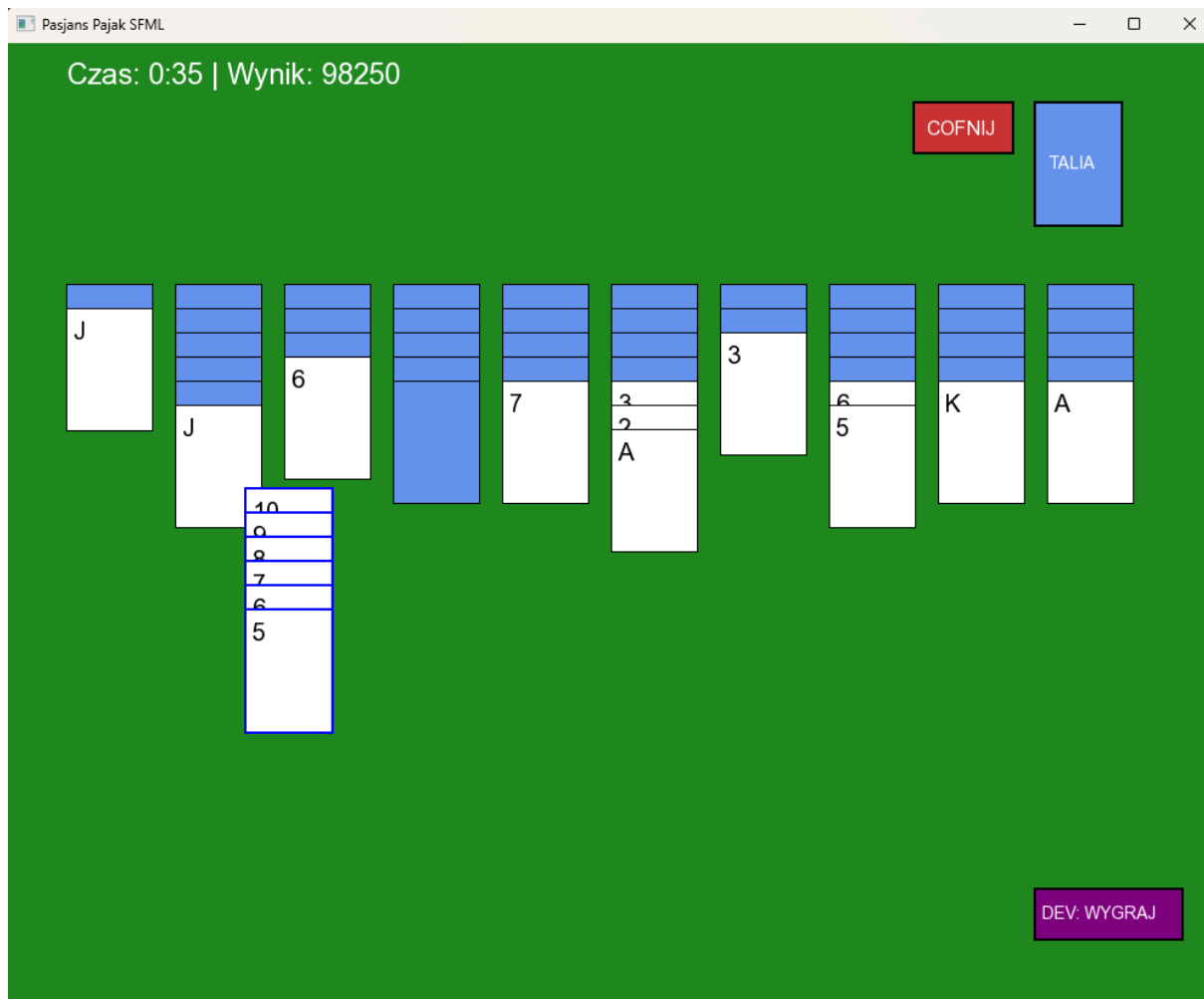
- **Karty:** Obiekty graficzne posiadające dwa stany: zakryty (rewers) i odkryty (awers z wartością).
- **Kolumny robocze:** 10 stosów, na których gracz operuje kartami.
- **Talia pomocnicza (Stock Pile):** Stos kart w prawym górnym rogu służący do rozdawania nowych rzędów.
- **Obszar sekwencji zebranych:** Miejsce w lewym górnym rogu, gdzie trafiają ukończone sekwencje (Król-As).
- **Interfejs HUD:** Tekstowe wyświetlacze czasu gry oraz aktualnego wyniku punktowego.

Sposób interakcji:

- **Sterowanie:** Wyłącznie za pomocą myszy (Drag & Drop). Kliknięcie LPM na kartę "podnosi" ją wraz z legalnym stosem pod nią.
- **Kolizje:** Logiczne sprawdzanie współrzędnych kursora względem obszarów kolumn przy upuszczaniu karty.
- **Punktacja:** Każda ukończona sekwencja dodaje punkty. Czas gry wpływa negatywnie na wynik zgodnie ze wzorem:
$$\text{Wynik} = 100\,000 - (t * 50).$$

2. Lista funkcjonalności i podział prac

Mechanika	Osoba odpowiedzialna
Silnik logiczny : Zarządzanie taliami, tasowanie, sprawdzanie poprawności sekwencji.	Piotr
System Drag & Drop : Obsługa zdarzeń myszy, przesuwanie sprite'ów, wykrywanie kolumn.	Maciej
Zarządzanie pamięcią : Wykorzystanie <code>std::unique_ptr</code> w kontenerze polimorficznym.	Obaj (kluczowa funkcja)
System Undo : Możliwość cofnięcia ruchu (snapshoty stanu gry).	Piotr
UI i Animacje : Wyświetlanie czasu, punktacji, menu końcowego.	Maciej
Zapis/Odczyt : Tabela wyników zapisywana do pliku .txt.	Maciej



```
// Fragment pętli rysującej karty w kolumnach
for (int i = 0; i < 10; ++i) {
    for (size_t j = 0; j < stosy[i].size(); ++j) {
        // Pomijamy rysowanie kart, które są aktualnie przeciągane myszką
        if (isDragging && i == dragCol && j >= (size_t)dragRow) continue;

        sf::RectangleShape rect(sf::Vector2f(szer, wys));
        rect.setPosition(50.f + i * offX, startY + j * offY);
        rect.setFillColor(stosy[i][j].odkryta ? sf::Color::White : sf::Color(100, 149, 237));
        window.draw(rect);
    }
}

if (isDragging) {
    for (size_t k = (size_t)dragRow; k < stosy[dragCol].size(); ++k) {
        float x = mousePos.x - dragOffset.x;
        float y = mousePos.y - dragOffset.y + ((k - dragRow) * offY);

        sf::RectangleShape dRect(sf::Vector2f(szer, wys));
        dRect.setPosition(x, y)
        window.draw(dRect); } }
```

Dziękuję za uwagę